

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1. In the lecture, why did three different teams building the flower identification app get such different results even though they used the same model architecture?

1 / 1 point

- ☐ They trained for different numbers of epochs
- ☒ They handled and prepared the data differently
- ☐ They used different loss functions
- ☐ They used different batch sizes

✔ Nice work

Yes. Even with "the exact same neural network architecture—identical layers, same parameters," the teams got wildly different results because "bad data handling means bad results." How they accessed and prepared the Oxford Flowers dataset made all the difference.

2. Three categories of data pipeline problems are introduced: access, quality, and efficiency. Which category involves loading data in batches rather than one item at a time?

1 / 1 point

- ☐ Access problems
- ☐ Quality problems
- ☒ Efficiency problems
- ☐ Architecture problems

✔ Nice work

Yes. Efficiency problems occur when "you need to load everything efficiently, in batches—not one image at a time—or your training will grind to a halt."

3. In the `__init__` method of the Oxford Flowers dataset, why are image paths stored but the actual images not loaded into memory?

1 / 1 point

- ☐ Loading images in `__init__` would be too slow
- ☒ Loading all images at once would use too much memory
- ☐ PyTorch doesn't allow loading images in `__init__`
- ☐ Images must be transformed before loading

✔ Nice work

Yes. This approach is called "lazy loading"—you store paths and only load images when needed in `__getitem__`. The lecture emphasizes this is "critical" because loading all 8,189 images at once could use gigabytes of memory.

4. The Oxford Flowers dataset labels range from 1 to 102, but PyTorch expects labels from 0 to 101. Where should you fix this indexing issue?

1 / 1 point

- ☐ In the `__getitem__` method when returning each label
- ☒ In the `__init__` method by subtracting 1 from all labels
- ☐ In the `__len__` method when reporting dataset size
- ☐ PyTorch will automatically adjust the labels

✔ Nice work

Yes. The lecture shows fixing this during setup by subtracting 1 from every label in the array. This way, the correction happens once instead of thousands of times during training when `__getitem__` is called repeatedly.

5. Why does the Normalize transform help neural networks learn more effectively?

1 / 1 point

- ☐ It makes all images the same brightness
- ☒ It spreads pixel values for better learning
- ☐ It reduces the file size of images
- ☐ It converts images from PIL format to tensors

✔ Nice work

Yes. The lecture explains that without normalization, values might "cluster near 1" or "near 0," which "squeezes important detail into a narrow range." Normalization spreads values out, "making it easier for the network to learn what matters."

6. `ToTensor()` performs three important operations. Which of the following is NOT one of them?

1 / 1 point

- ☐ Rearranges dimensions to [channels, height, width]
- ☐ Scales pixel values from 0-255 to 0-1
- ☐ Converts the image from a PIL object to a PyTorch tensor
- ☒ Normalizes pixel values using mean and standard deviation

✔ Nice work

Yes. `ToTensor()` does NOT perform normalization with mean and standard deviation—that's what the separate `Normalize` transform does. `ToTensor()` only scales values from 0-255 to 0-1, which is different from normalization.

7. What's wrong with loading data from a large file inside the `__getitem__` method instead of in `__init__`?

1 / 1 point

- ☐ PyTorch doesn't allow file reading in `__getitem__`
- ☐ It prevents the `DataLoader` from shuffling the data
- ☒ The entire file is read for every sample
- ☐ It causes the data to be loaded into memory all at once and crash

✔ Nice work

Yes. The lecture shows this as a common mistake: "Every time you request one sample, it reads the entire file! With 5,732 images and batches of 32, that's reading your huge file 5,732 times per epoch." The fix is to "load once in `__init__`"

8. When you iterate through a `DataLoader`, what does each iteration give you?

1 / 1 point

- ☐ A single image and its label
- ☒ A batch of images and a batch of labels
- ☐ Just the images—you need to fetch labels separately
- ☐ The entire dataset at once

✔ Nice work

Yes. Each batch gives you two things: a batch of images and a batch of labels. For example, with `batch_size=32`, you get a tensor of shape `[32, 3, 224, 224]` for images and `[32]` for labels.

9. Why should you apply data augmentation to the training set but not to the validation set?

1 / 1 point

- ☐ Augmentation is too slow to apply during validation
- ☐ PyTorch doesn't support augmentation on validation data
- ☒ Validation needs consistent data to measure improvements
- ☐ Augmentation only works on images the model has already seen

✔ Nice work

Yes. You want to evaluate your model on consistent data. If validation images changed randomly too, you couldn't tell whether performance changes were due to real model improvements—or just different inputs. Training benefits from variation; evaluation requires stability.

10. In the robust dataset implementation, what happens when `getitem` encounters a corrupted or problematic image?

1 / 1 point

- ☐ It raises an exception and stops training
- ☐ It returns `None` and continues to the next batch
- ☒ It logs the error and retrieves another image
- ☐ It replaces the corrupted image with a blank tensor

✔ Nice work

Yes. The lecture shows the strategy: Log exactly what went wrong, print a warning, and gracefully skip to the next image. That recursive call to `getitem` keeps your pipeline moving—even when files are broken. This prevents crashes while maintaining a record of issues.

